

Advanced Programming Language

Progetto Scopone Scientifico

Vincenzo Rubulotta 1000034149

Paolo Alfò 1000030219

1. Introduzione e Architettura di Sistema	3
1.1 Panoramica del Progetto	3
1.2 Struttura delle Directory e Organizzazione del Codice	3
2. Interfaccia di Comunicazione: gRPC e Protobuf	5
2.1 Nota Metodologica: Stub Auto-Generati	5
2.2 Analisi del File messaggi_partita.proto	6
3. Backend GO	7
3.1 Librerie e Dipendenze Principali	7
3.2 Struttura del Modulo	7
3.3 Focus Implementativo	8
4. Frontend TUI C#	10
4.1 Librerie e Dipendenze Principali	10
4.2 Struttura del Modulo	10
4.3 Focus Implementativo	11
5. Data Analytics Python	12
5.1 Librerie e Dipendenze	12
5.2 Struttura del Modulo	12
5.3 Focus Implementativo	12
6. Containerizzazione Docker	14
6.1 Dockerfile	14
6.2 Orchestrazione Docker Compose	14

1. Introduzione e Architettura di Sistema

1.1 Panoramica del Progetto

Il presente elaborato descrive la progettazione e l'implementazione di un sistema distribuito per la gestione e la giocabilità del tradizionale gioco di carte "Scopone Scientifico".

L'infrastruttura si avvale della containerizzazione tramite Docker e dell'orchestrazione mediante Docker Compose, permettendo l'esecuzione fluida e standardizzata dei vari servizi indipendentemente dall'ambiente host.

Le tecnologie e i linguaggi di programmazione sono stati selezionati in base alle specifiche responsabilità di ciascun modulo:

Go (Golang): Scelto per lo sviluppo del Backend grazie alle sue eccellenti performance native e ai costrutti di concorrenza integrati (Goroutine e Mutex), fondamentali per la gestione asincrona dei giocatori CPU e la sincronizzazione dello stato della partita.

C# (.NET): Utilizzato per lo sviluppo del Client e dell'interfaccia utente testuale (TUI), offrendo un ambiente robusto e fortemente tipizzato per la gestione degli stream di dati e l'interazione con l'utente finale.

Python: Impiegato per il modulo di Data Analytics, sfruttando le librerie standard di settore (Pandas, Matplotlib) e il micro-framework Flask per l'elaborazione dei dati storici e la generazione dinamica di dashboard statistiche.

La comunicazione inter-processo tra il client e il server di gioco è demandata al framework gRPC basato su Protocol Buffers, il quale supporta nativamente architetture di rete in streaming bidirezionale.

1.2 Struttura delle Directory e Organizzazione del Codice

Il workspace del progetto è stato organizzato in modo modulare. Di seguito viene presentata la panoramica ad alto livello dell'albero delle directory root:

```
PROGETTO_APL/  
├── Analytics/  
├── Proto/  
├── ScoponeClient/  
├── Server_Go/  
├── docker-compose.yaml  
└── README.md
```

Analytics/: Directory contenente il microservizio preposto all'analisi dei dati e alla generazione delle statistiche. L'intero modulo è sviluppato in linguaggio Python.

ScoponeClient/: Directory contenente il codice sorgente del frontend, ovvero il client di gioco con interfaccia testuale (TUI), interamente implementato in C#.

Server_Go/: Directory dedicata al core engine del sistema, contenente la logica di backend, la validazione delle regole e la gestione della concorrenza, sviluppato in Go.

docker-compose.yaml: File di configurazione per l'orchestrazione dei container, responsabile della definizione, del build e dell'avvio simultaneo del backend e del modulo di analytics.

README.md: File di documentazione contenente le istruzioni operative per l'installazione, l'esecuzione dell'applicativo e la mappatura delle interfacce API.

Proto/: Questa directory contiene il file originario .proto e i relativi file pre-compilati (stub generati automaticamente dal compilatore protoc per i linguaggi target C# e Go). Sebbene i file generati siano stati successivamente copiati all'interno delle rispettive directory operative (ScoponeClient e Server_Go) per l'effettiva compilazione dei moduli, si è scelto di mantenere questa cartella isolata nella root. Tale scelta progettuale è finalizzata a fornire una visione chiara della struttura Protobuf di partenza e dei suoi artefatti, isolando il contratto di comunicazione dalla logica applicativa vera e propria.

2. Interfaccia di Comunicazione: gRPC e Protobuf

La comunicazione bidirezionale tra il frontend (C#) e il backend (Go) è interamente definita e gestita tramite il framework gRPC e il linguaggio di serializzazione Protocol Buffers (Protobuf). Tutta la definizione del contratto di comunicazione risiede all'interno della directory Proto, che contiene il file sorgente “messaggi_partita.proto” e due sottocartelle (“out_go” e “out_csharp”) destinate a ospitare il codice generato.

2.1 Nota Metodologica: Stub Auto-Generati

È importante precisare che i file sorgente C# e Go generati a partire dal file .proto sono stati prodotti in modo completamente automatizzato dal compilatore protoc.

Tali file espongono i metodi e le interfacce necessarie per la comunicazione di rete, ma la loro implementazione interna è gestita dal toolchain di gRPC e non costituisce codice applicativo scritto direttamente dai membri del team. Pertanto, ai fini di questa relazione e dell'architettura del progetto, gli stub generati contenuti nelle directory out_go e out_csharp sono da intendersi a tutti gli effetti come librerie esterne.

Per completezza, si riportano di seguito i comandi eseguiti per la generazione automatica degli stub:

Compilazione per il Client C#:

```
dotnet new console -n TempGetPlugin
cd TempGetPlugin
dotnet add package Grpc.Tools
cd ..
rm -rf TempGetPlugin
PLUGIN_PATH=$(find ~/.nuget/packages/grpc.tools -name "grpc_csharp_plugin"
               | grep "macosx" | head -n 1)
protoc --proto_path=. --csharp_out=./out_csharp --csharp_opt=file_extension=.g.cs
       --grpc_out=./out_csharp
       --plugin=protoc-gen-grpc=$PLUGIN_PATH messaggi_partita.proto
```

Compilazione per il Server Go:

```
export PATH="$PATH:$(go env GOPATH)/bin"
protoc --proto_path=. --go_out=./out_go --go_opt=paths=source_relative
       --go-grpc_out=./out_go
       --go-grpc_opt=paths=source_relative messaggi_partita.proto
```

2.2 Analisi del File messaggi_partita.proto

Il file “messaggi_partita.proto” funge da contratto formale tra i microservizi.

Invece di affidare la comunicazione a formati testuali generici e non tipizzati (come JSON o XML over HTTP), si è scelto di definire a priori le strutture dati e le procedure di rete.

Tramite l'uso di Enumeratori (per definire elementi immutabili come i semi, i valori delle carte e i ruoli dei giocatori al tavolo) e di messaggi strutturati, il file mappa l'intero dominio del gioco dello Scopone Scientifico.

Queste strutture definiscono il payload esatto che viaggerà in rete. Il vantaggio architetturale è che il client C# e il server Go condividono le stesse classi native senza ambiguità ottenendo quindi una forte tipizzazione.

La sezione service definisce le procedure remote (RPC) che il client può invocare sul server. Dal punto di vista progettuale, le API sono state modellate sfruttando due paradigmi di comunicazione offerti da gRPC:

RPC Unarie (Modello Request/Response): Utilizzate per le interazioni classiche e puntuali, come l'inizializzazione della partita, la giocata di una singola carta da parte dell'utente o la richiesta di calcolo del punteggio a fine smazzata.

Server-Side Streaming RPC: Utilizzata per la fase di osservazione (observe_turn). Invece di forzare il client a effettuare un continuo polling per scoprire se la CPU ha giocato, il client apre uno stream di ascolto passivo. Il server Go ha così la responsabilità di effettuare il push degli aggiornamenti del tavolo in tempo reale verso il frontend non appena i bot effettuano una mossa.

Il frontend C# quindi si limita a renderizzare uno stato calcolato altrove, mentre l'intera complessità delle regole, dei turni rimane isolata all'interno del server Go.

3. Backend GO

Il core engine del sistema è interamente sviluppato in linguaggio **Go (Golang)**. La scelta di questa tecnologia è motivata dalla sua naturale predisposizione alla programmazione di rete e dalla gestione efficiente della concorrenza tramite Goroutine.

Il server ha la responsabilità di validare le mosse, calcolare i punteggi, gestire le intelligenze artificiali (bot) e salvare i dati per il modulo di Analytics.

3.1 Librerie e Dipendenze Principali

Le principali librerie impiegate sono:

google.golang.org/grpc e net: Utilizzate per esporre il servizio sulla porta TCP 50051 e implementare le interfacce gRPC autogenerate, gestendo il ciclo di vita delle richieste e lo streaming dati.

sync: Fondamentale per l'importazione dei Mutex, necessari a prevenire *Race Condition* e garantire l'accesso in mutua esclusione alle strutture dati condivise.

encoding/csv e os: Impiegate per l'apertura in modalità append e la scrittura non bloccante dei log di partita sul file system condiviso con il modulo Python.

math/rand e time: Utilizzate per la generazione univoca dei Game_ID (basata su timestamp UNIX) e per l'algoritmo di mescolamento del mazzo di carte all'avvio di ogni smazzata.

context: È stata impiegata per la gestione del ciclo di vita delle RPC, permettendo di incapsulare lo stato della richiesta e gestire in sicurezza la concorrenza e gli stream di rete.

fmt, log, strconv: Impiegate trasversalmente per le operazioni di I/O. Nello specifico, log e fmt garantiscono il tracciamento formattato degli eventi a terminale, mentre strconv fornisce primitive sicure per la conversione dei tipi di dato.

3.2 Struttura del Modulo

Il codice sorgente è organizzato nei seguenti file per garantire il principio di singola responsabilità:

main.go: Entry point dell'applicazione. Inizializza il listener TCP, istanzia il server gRPC, vi registra l'interfaccia go_backend e lo pone in ascolto.

grpc_server.go: Contiene le implementazioni effettive delle Remote Procedure Call (es. **StartGame**, **PlayCard**, **ObserveTurn**, **CalcolaPunteggio**). Questo file funge da "ponte", traducendo i messaggi di rete in chiamate alle funzioni logiche interne.

logica_di_gioco.go Contiene la struttura centrale **GameSession** e tutta la complessa logica delle regole dello Scopone. Qui risiedono gli algoritmi per la risoluzione delle combinazioni di carte a terra (**trovaCombinazioni**), il controllo della scopa, la gestione del tavolo (**tableManager**), il passaggio dei turni e il calcolo dei punti di fine round (**calcolaRisultati**).

utility_function.go: Modulo di supporto contenente funzioni come il calcolo del valore matematico delle singole combinazioni e l'implementazione fisica dell'I/O per il salvataggio delle metriche sui file CSV.

3.3 Focus Implementativo

Passiamo quindi alla descrizione delle scelte implementative del backend per la gestione della concorrenza, progettata per supportare istanze di gioco multiple e l'esecuzione asincrona delle “intelligenze artificiali”.

All'avvio di una nuova partita (**StartGame**), il server istanzia tre distinte Goroutine in background, una per ciascun bot controllato dalla CPU (**runCPUPlayers**). Ogni bot non agisce tramite un ciclo procedurale bloccante, ma rimane in ascolto asincrono su un proprio canale. Quando lo stato della partita cambia e il turno passa a un bot, la relativa Goroutine analizza il tavolo tramite la funzione **calcolaGiocata**, ed esegue la mossa reimmettendo un aggiornamento nel sistema. Questo disaccoppia completamente le tempistiche della CPU da quelle dell'utente.

Il server implementa una variante concorrente del Pattern Observer. La funzione **subscribe()** crea un channel dedicato per chiunque voglia osservare la partita (sia il client tramite la stream gRPC **ObserveTurn**, sia i bot CPU). Quando un qualsiasi giocatore effettua una giocata valida, la funzione **broadcastUpdate()** itera su tutti i canali registrati e invia il nuovo stato del tavolo, garantendo un aggiornamento real-time e reattivo per tutti i partecipanti.

Poiché gRPC esegue ogni richiesta RPC in una propria goroutine separata, l'accesso simultaneo ai dati potrebbe corrompere la memoria. Per evitare Race Condition, sono stati implementati due livelli di locking:

managerMU: Protegge la mappa condivisa **gameManager**, impedendo crash di sistema qualora due client provino a creare o cercare una partita nello stesso identico millisecondo.

GameSession.mu: Ogni singola partita possiede un proprio Mutex indipendente. Durante le fasi critiche ,come la verifica del giocatore di turno, l'aggiornamento del tavolo o la transizione di stato, il Mutex viene bloccato (Lock). Questo assicura che se un bot e un utente provano ad agire simultaneamente, le loro mosse vengano processate in rigoroso ordine sequenziale, mantenendo lo stato del gioco sempre coerente e matematicamente esatto.

Si vuole infine dare una delucidazione, su come è stata implementata la gestione delle scelte per il calcolo delle possibili prese a terra.

Il sistema implementa un algoritmo di ricerca in profondità(DFS). L'algoritmo esplora l'albero delle combinazioni delle carte presenti sul tavolo, isolando tutti i sottoinsiemi la cui somma dei valori eguaglia esattamente il valore della carta giocata.

Qualora l'algoritmo individui molteplici combinazioni valide la logica di gioco si biforca:

Se a giocare è la CPU, interviene una funzione euristica di valutazione (**calcolaPunteggioCombinazione**) che soppesa il valore delle carte e sceglie autonomamente la presa più vantaggiosa.

Se a giocare è l'utente umano, il server sospende il calcolo del turno, inserisce le opzioni disponibili in un array di Option e restituisce al client un payload con il flag `ConflictResolutionNeeded = true`. In questo modo, il potere decisionale viene rigorosamente delegato al frontend e all'utente finale.

4. Frontend TUI C#

L'interfaccia utente del sistema è implementata tramite un client testuale (TUI) sviluppato in C# su piattaforma .NET. La scelta di una TUI a riga di comando permette di mantenere il client estremamente leggero, focalizzato puramente sulla logica di interazione, delegando l'intera complessità computazionale al backend.

4.1 Librerie e Dipendenze Principali

Il client fa uso del seguente set di librerie, pacchetti esterni per la rete e librerie standard .NET per la gestione dell'asincronia e del terminale:

Grpc.Net.Client e Google.Protobuf: Costituiscono il core della comunicazione di rete. Forniscono il client HTTP/2 necessario per instaurare il canale gRPC (**GrpcChannel.ForAddress**) e deserializzare i messaggi Protobuf in oggetti nativi C#.

System.Threading.Tasks: Fondamentale per la programmazione asincrona (**async/await e Task.Run**), permette di gestire flussi di rete I/O-bound senza bloccare il thread principale dell'interfaccia utente.

System.Diagnostics: Utilizzata per l'invocazione del metodo **Process.Start**, che permette l'integrazione con il sistema operativo host per l'apertura dinamica del browser web (necessaria per la visualizzazione delle dashboard di Analytics).

System.Text: Impiegata per forzare la codifica UTF8 sul terminale (**Console.OutputEncoding = Encoding.UTF8**), fondamentale per il corretto rendering dei caratteri Unicode.

4.2 Struttura del Modulo

Il codice sorgente segue un paradigma architetturale orientato alla separazione delle responsabilità:

MessaggiPartita.cs e MessaggiPartitaGrpc.cs: Come specificato nel Capitolo 2, questi file sono gli stub auto-generati dal compilatore protoc. Vengono inclusi nel progetto per fornire le classi C# dei messaggi e il client tipizzato **go_backendClient**.

Program.cs: L'entry-point dell'applicazione. Si occupa del bootstrap, della connessione al server sulla porta 50051, dell'autenticazione iniziale e dell'orchestrazione dei thread. Contiene il loop principale per la cattura dell'input da tastiera (**GestisciInputUtente**) e le logiche di invio delle RPC.

gameState.cs: Implementa lo stato centralizzato del client. Mantiene in memoria le carte in mano, quelle sul tavolo, i punteggi e il log dei messaggi. È progettato per essere una classe che viene aggiornata dai dati provenienti dal server.

TuIGraphics.cs: Riceve uno snapshot dello stato attuale e si occupa di tradurla in coordinate fisiche sullo schermo tramite il posizionamento del cursore (**Console.SetCursorPosition**) e la manipolazione dei colori (**Console.BackgroundColor**).

4.3 Focus Implementativo

La sfida tecnica principale del client consisteva nel mantenere l'interfaccia grafica costantemente aggiornata con le mosse della CPU, permettendo al contempo all'utente umano di navigare tra le proprie carte senza interruzioni o blocchi dell'interfaccia.

Questo problema è stato risolto tramite un'architettura multi-thread basata sull'ascolto passivo:

Una volta inizializzata la partita, il Program.cs delega a un thread in background (**Task.Run() => AscoltaServer()**) la gestione dello stream gRPC **ObserveTurn**. All'interno di questo task, il client itera in modo asincrono sui messaggi in arrivo dal server (**await stream.ResponseStream.MoveNext()**). Ogni qualvolta la CPU o un altro attore effettua una mossa nel backend Go, il flusso si sblocca, il client riceve il **TurnUpdate** ed elabora visivamente la carta giocata e le eventuali prese.

Poiché lo stato del gioco viene letto dal thread principale e contemporaneamente modificato dal thread in background, si verificherebbero inevitabilmente collisioni di memoria. Per prevenire questo fenomeno, ogni metodo che legge o altera lo stato (**SpostaCartaSulTavolo**, **GetSnapshot** o **ResetNewRound** ecc...) è racchiuso in un blocco lock (**_stateLock**). Questa mutua esclusione locale assicura un rendering privo di conflitti e una consistenza rigorosa dei dati mostrati a video.

Si vuole anche descrivere la decisione implementativa attuata per l'invio della giocata. Quando la RPC restituisce il flag **ConflictResolutionNeeded = true**, il client intercetta l'eccezione logica. Invece di far avanzare il turno, la TUI congela il gioco, decodifica il vettore **Option** ricevuto dal server e formatta a video le possibili combinazioni di presa. Viene quindi innescato un oop di input che attende la scelta numerica dell'utente prima di inviare nuovamente al server la chiamata **play_card** con la combinazione bersaglio esplicita (**TargetCard**), delegando sempre al server la validazione finale.

5. Data Analytics Python

Questo modulo è implementato come un server web indipendente scritto in Python, il cui compito è elaborare i log di sistema ,scritti dal backend Go, e generare dashboard analitiche visualizzabili direttamente nel browser dell'utente.

5.1 Librerie e Dipendenze

Le librerie impiegate sono:

Flask: Un micro-framework web leggero e flessibile. È stato scelto per esporre rapidamente gli endpoint HTTP (API REST-like).

Pandas: Permette di caricare i file CSV all'interno di DataFrame ottimizzati, facilitando operazioni di filtraggio, raggruppamento e calcolo delle metriche (es. medie, conteggi booleani e percentuali di vittoria).

Matplotlib: Motore di rendering grafico utilizzato per la generazione degli istogrammi, dei grafici a torta e delle tabelle riassuntive.

io: Modulo standard di Python, usato per la gestione dei buffer binari in RAM (BytesIO), evitando I/O superfluo sul disco durante il rendering delle immagini.

5.2 Struttura del Modulo

A differenza dei moduli precedenti, il microservizio di Analytics è progettato intenzionalmente come un'applicazione single-file. Il file definisce le costanti di percorso ai volumi condivisi (/data/match_history.csv e /data/rounds_history.csv) ed espone due route HTTP principali:

/stats/match/<game_id>: Rotta parametrica dedicata all'analisi di una singola sessione di gioco.

/stats: Rotta globale per l'aggregazione storica di tutte le partite giocate sul server.

5.3 Focus Implementativo

Poiché il codice Python viene eseguito all'interno di un container Docker privo di server grafico, l'uso standard di Matplotlib causerebbe il crash dell'applicativo al momento del disegno. Per prevenire ciò, il backend di Matplotlib è stato forzato sulla modalità Agg (Anti-Grain Geometry). Questa scelta implementativa forza la libreria a renderizzare le immagini in background, senza tentare di aprire finestre grafiche sul sistema operativo host.

Quando il client effettua una richiesta HTTP, il server Flask carica i dati dal volume condiviso. Tramite Pandas, i dati grezzi vengono puliti e formattati (es. conversione 'true'/'false' in interi booleani per il conteggio dei punti). La separazione logica garantisce che il server Go non sprechi cicli di CPU per aggregare le statistiche, delegando il lavoro al container Python.

Una volta generati i sub-plot, la figura non viene salvata fisicamente sul disco rigido del container. L'immagine viene serializzata direttamente nella memoria volatile (RAM) tramite un buffer **io.BytesIO()**. Flask utilizza infine la funzione **send_file** per inserire il payload binario direttamente nella risposta HTTP con il MIMEType **image/png**.

Non scrivendo su disco, si massimizzano le prestazioni di risposta e si previene il riempimento dello storage del server con file temporanei.

Terminata la richiesta HTTP, il buffer viene automaticamente deallocato dal Garbage Collector di Python, mantenendo il microservizio stateless e scalabile.

6. Containerizzazione Docker

Per garantire la massima portabilità, isolamento degli ambienti e riproducibilità del sistema, l'intero strato di backend e analytics è stato pacchettizzato utilizzando la tecnologia di containerizzazione Docker. L'orchestrazione dei container è gestita in modo dichiarativo tramite Docker Compose.

6.1 Dockerfile

La pacchettizzazione dei due microservizi segue due paradigmi distinti, ottimizzati per le specificità dei rispettivi linguaggi di programmazione.

Il Dockerfile relativo al motore di gioco (GO), è stato scritto in modo tale da garantire la separazione dell'ambiente di compilazione dall'ambiente di esecuzione:

Fase di Build: Utilizza l'immagine ufficiale completa golang:1.25.4. All'interno di questo ambiente vengono scaricate le dipendenze e il codice viene compilato.

Fase di Run: Il binario così generato viene copiato in una nuova immagine basata su alpine:latest ed viene successivamente esposta la porta TCP 50051 per gRPC.

Essendo Python un linguaggio interpretato, non è possibile applicare un vero e proprio multi-stage build per i binari. Si è optato quindi per l'immagine base python:3.10-slim. Questa rimuove compilatori e pacchetti di sistema superflui, offrendo inoltre il mantenimento delle librerie essenziali per l'installazione di pacchetti complessi come Pandas e Matplotlib tramite pip. Il container espone infine la porta 5001 per la ricezione delle richieste HTTP.

6.2 Orchestrazione Docker Compose

Il file docker-compose.yaml definisce due servizi principali (scohone-backend e scohone-analytics) e ne orchestra il ciclo di vita, la comunicazione e implementa le seguenti caratteristiche fondamentali per la gestione dell'interno progetto:

Dipendenze di Avvio: Il servizio di analytics dichiara una dipendenza esplicita dal backend (depends_on: - scohone-backend). Questo assicura un ordine di avvio deterministico: il container Python non verrà avviato finché il motore di gioco Go non sarà operativo.

Isolamento delle Porte: Il compose mappa le porte interne dei container sulle porte dell'host (50051:50051 per gRPC e 5001:5001 per HTTP). Questo permette al client C# di interrogare i due microservizi come se fossero in esecuzione in locale.

Condivisione dello Stato: La comunicazione tra i due servizi avviene tramite il volume

stats_volume. Questo volume Docker viene montato nella directory /data di **entrambi** i container. Quando il server Go calcola le metriche a fine partita, scrive i risultati nei file CSV all'interno di /data.

Il server Python, avendo accesso in lettura al medesimo volume condiviso, può elaborare e graficare i dati in tempo reale senza dover instaurare connessioni di rete per il recupero delle metriche.